Multi-View Flow Representation and Relation Graph Modeling for Network Traffic Classification

Huanhuan Gu*, Qingjun Xiao†, *Member, IEEE*, Mingyi Wang‡, Jiaming Zhang†, Qianmu Li*
*School of Cyber Science and Engineering, Nanjing University of Science and Technology, Wuxi 214400, Jiangsu, China
†School of Cyber Science and Engineering, Southeast University, Nanjing 211189, Jiangsu, China
‡Nanjing Sinovatio Technology Co., Ltd., Nanjing 210000, Jiangsu, China
guhuanhuan@njust.edu.cn, csqjxiao@seu.edu.cn, sinovatio2007@163.com, jmingzhang@seu.edu.cn, qianmu@njust.edu.cn

2026 IEEE/ACM International Symposium on Quality of Service (IWQoS) | 979-8-3195-1703-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/IWQoS70441.2026.11661067

***Abstract*—The widespread deployment of encryption protocols has fundamentally reduced payload visibility in network traffic, posing significant challenges to network behavior inference systems (e.g. Network Intrusion Detection System, NIDS) based solely on packet contents. Existing approaches often depend on coarse-grained flow-level statistics or isolated protocol metadata, which limits their ability to capture fine-grained TCP/IP packet sequences within individual flows as well as interaction patterns across multiple flows. From passive traffic measurements, we observe that an encrypted client–server communication typically spans multiple protocol stages, including domain name resolution, TLS handshake, and encrypted payload transmission. Additionally, complex network behaviors are often manifested through a sequence of correlated five-tuple flows rather than a single flow.**

To address these challenges, we propose a robust network behavior modeling framework, MV-FlowGraph, which characterizes encrypted traffic by jointly modeling multi-view intra-flow features and temporal inter-flow dependencies. Specifically, MV-FlowGraph consists of three key components: (1) a flow-level feature extraction and representation module that leverages server-side domain features recovered from plaintext DNS traffic, TLS handshake metadata, and TCP/IP packet transmission sequences; (2) a flow relation graph module that builds application-level flow graphs to capture inter-flow concurrency and sequential dependencies; and (3) a multi-view fusion module that employs BERT, autoencoders, and Graph Attention Networks to jointly model multi-view intra-flow features and temporal inter-flow features. Extensive experimental results verify that MV-FlowGraph outperforms state-of-the-art methods (e.g., AppScanner, FS-Net, and FG-Net) in detecting network anomalies targeting online services.

I. Introduction

The rapid evolution of network technologies and applications has resulted in increasingly complex and large-scale network traffic, making accurate traffic analysis and behavior characterization a fundamental challenge for modern networks. Network administrators rely heavily on traffic measurements to understand communication patterns, detect abnormal behaviors, and ensure the robustness and dependability of networked systems [1]. In practice, many critical network behaviors, such as service abuse, scanning activities, and large-scale resource consumption, are manifested through subtle changes in traffic patterns rather than explicit payload contents, which motivates the need for more fine-grained traffic modeling techniques.

Corresponding author: Qianmu Li (email: qianmu@njust.edu.cn).

979-8-3195-1703-6/26/$31.00 ©2026 IEEE

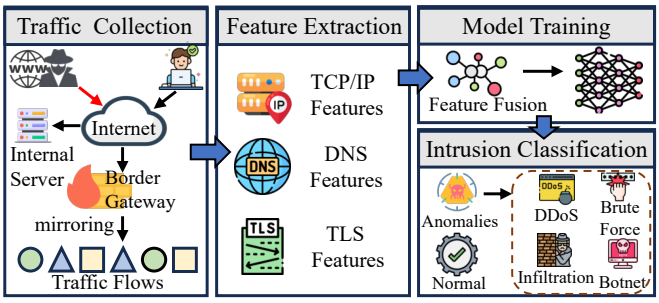


Fig. 1: Overview of multi-view flow representation and relation graph modeling framework for encrypted traffic analysis.

However, the widespread adoption of encryption protocols (e.g., TLS 1.2 and 1.3), together with increasingly complex traffic behaviors, poses significant challenges to reliable network behavior inference. Traditional Deep Packet Inspection (DPI) methods [2], which rely on pattern matching over plaintext payloads, are rendered ineffective by encryption. Although machine-learning-based approaches that exploit side-channel information from network flows (e.g., statistical features) have emerged as viable alternatives, they still suffer from critical accuracy limitations in modern encrypted network environments.

First, many existing works rely on simple statistical features extracted from individual network flows [3], [4], overlooking the richer information within directed IP packet sequences. Second, they make limited use of protocol metadata, such as DNS queries and TLS handshake fields (e.g., domain names, cipher suites, and certificate chains) that precede encrypted payload transmission. Third, most prior methods lack effective mechanisms for multi-view feature fusion and tend to model network flows from a single perspective, such as flow statistics [5] or directed packet sequences [6], instead of integrating complementary statistical features, packet-level dynamics, and protocol semantics into a unified representation. Finally, they fail to capture relations among multiple flows within the same client-server session. Many complex network behaviors (e.g., large-scale flooding or scanning) manifest as sequences of related flows between clients and servers, which are hard to accurately characterize when flows are analyzed in isolation.

To address these challenges, we propose *MV-FlowGraph*, a multi-view flow and relation graph modeling framework for

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

analyzing encrypted network traffic. As shown in Fig. 1, MV-FlowGraph collects traffic flows at the border gateway and extracts heterogeneous features from multiple perspectives, including TCP/IP flow statistics, DNS metadata, and TLS handshake attributes. These multi-view features are jointly fused to learn expressive flow representations. Beyond intra-flow feature modeling, MV-FlowGraph further incorporates temporal dependencies among multiple flows to capture session-level behavioral patterns. This design enables accurate and fine-grained characterization of network behaviors under highly imbalanced traffic distributions. During deployment, incoming traffic is processed in real time to support traffic analysis tasks, including the identification of abnormal or suspicious communication patterns in encrypted network environments.

Specifically, MV-FlowGraph comprises three key modules:

- **Intra-flow Feature Extraction Module**: This module extracts conventional IP-level packet statistics together with enriched DNS and TLS metadata (e.g., domain names, certificate subjects and issuers, and cipher suites), providing discriminative signals that are difficult for attackers to forge.
- **Multi-View Feature Fusion Module**: This module encodes flow-level statistical features by an autoencoder, models packet transmission sequences with a Transformer, and represents DNS/TLS textual payloads by a pre-trained BERT. The resulting representations from different views are then fused to produce enriched embeddings for individual flows.
- **Inter-Flow Relation Graph Modeling Module**: This module aggregates five-tuple flows into client–server sessions and constructs inter-flow relation graphs by explicitly modeling concurrent and sequential flow interactions according to their arrival times. Each session-level multi-flow relation graph is subsequently modeled using a graph neural network and classified to predict the corresponding behavioral labels.

The main contributions of this paper are summarized below.

- To enrich the representation of individual flows, we extract flow-level statistical features and directed packet-length sequences, together with DNS and TLS handshake metadata.
- To capture multi-flow behaviors, we group flows into client-server sessions, partition time-adjacent flows into bursts, link bursts via sequential relations to form flow relation graphs.
- Our architecture extracts intra-flow semantics from multiple views, i.e., flow-level statistical features, packet-level time series, and payload metadata are encoded by an autoencoder, a Transformer, and BERT, respectively. The resulting multi-view embeddings for each flow are then fused to form the vertex features in a flow relation graph, modeled by a GAT.
- Extensive experiments on multiple public datasets show that *MV-FlowGraph* achieves state-of-the-art performance and exhibits high robustness in network intrusion detection tasks.

II. Related work

In this section, we categorize existing network traffic classification methods based on the types of features they exploit. We then analyze the limitations of these approaches to motivate the design of our MV-FlowGraph framework for fusing multi-view features, and highlight its key contributions.

A. Flow-Level Feature Engineering-Based Methods

Existing network traffic classification methods rely on feature engineering. Early approaches focused on TLS handshake inspection, exploiting plaintext metadata such as cipher suites, certificate chains, and Service Name Indication (SNI). For instance, TLSmell [7] correlates Zeek logs to extract 33 features spanning flow statistics and certificate attributes for classifiers like SVM and CNN. Similarly, Lokoč et al. [8] utilize high-dimensional statistical fingerprints augmented with handshake duration, while Dai et al. [9] apply mutual information to filter weakly correlated TLS features. However, these methods are increasingly hindered by the widespread adoption of TLS 1.3, which encrypts most handshake extensions, and are vulnerable to obfuscation where attackers forge certificates.

To address TLS visibility limits, prior studies shifted toward encryption-resilient flow-level statistics, such as packet counts, sizes, and inter-arrival times. Methods like SBB-GP [5], AppScanner [10], ADABIND [11], FFP [3] and constraint-based semi-supervised methods [12] leveraged these features with machine learning to identify encrypted traffic. However, such coarse-grained statistical summaries fail to capture fine-grained temporal dynamics. To mitigate this, our *MV-FlowGraph* uses both 103-dimensional statistical features and packet-level time series to capture richer behavioral patterns.

B. IP Packet-Level Sequence Modeling Methods

To capture fine-grained temporal patterns, recent studies model network flows as time series of IP packet attributes using deep neural networks [13]. Early works employed CNNs and RNNs to extract sequential dependencies. For instance, DF [14] applies CNNs to packet direction sequences for Tor traffic fingerprinting, while FS-Net [6] utilizes bidirectional GRUs with a reconstruction mechanism to model packet length sequences. MGREL [15] further adopts a dual-granularity approach, fusing local handshake semantics via attention mechanisms with global packet sequences captured by LSTMs. However, these CNN- and RNN-based architectures often struggle to effectively capture long-range dependencies in prolonged connections, resulting in information loss [16].

Recently, Transformer-based architectures have shown strong capability in modeling long-range dependencies in network traffic. ET-BERT [16] learns contextualized representations from packet payload bytes through specialized pre-training, while TrafficFormer [17] captures long-term temporal dependencies in packet sequences via bidirectional time-frequency attention. However, the former method processes all raw packet payload bytes without explicitly distinguishing plaintext traffic from encrypted traffic, which incurs substantial computational overhead. In addition, both of them model each flow in isolation, overlooking inter-flow correlations that are critical for detecting multi-stage attacks. In contrast, our *MV-FlowGraph* extracts payload metadata only from plaintext traffic and encodes them with a pre-trained BERT model, as well as encodes packet time series by a Transformer. It also combines intra-flow sequence embeddings with inter-flow relation graphs to better characterize session-level behaviors.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

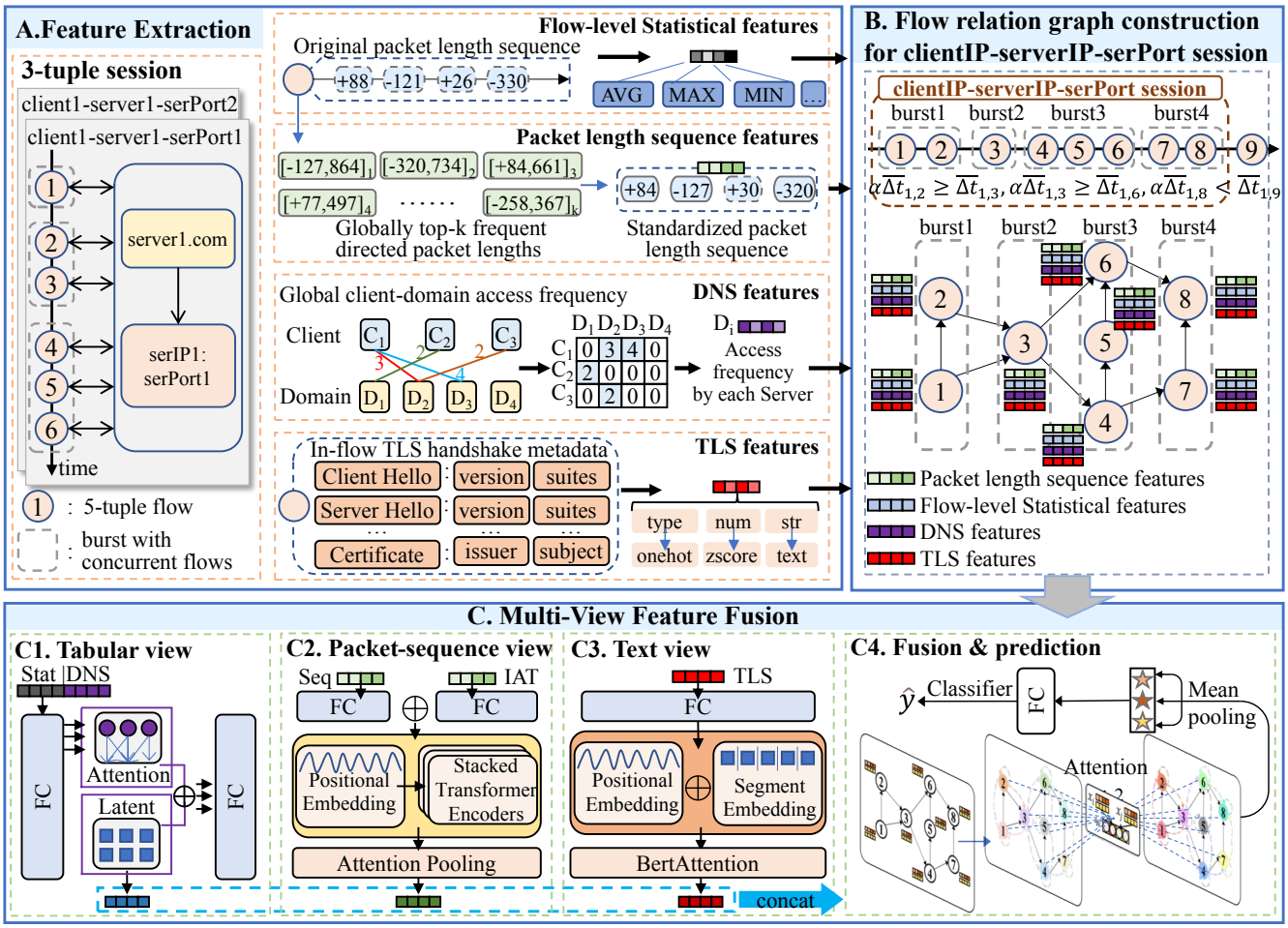


Fig. 2: MV-FlowGraph's data processing pipeline: (A) extracts features, (B) builds flow-relation graphs, and (C) trains models.

C. Session-Level Flow Relation Graph-Based Methods

Network behaviors are rarely confined to individual flows; many malicious activities, such as large-scale flooding and BruteForce attacks, often manifest as coordinated sequences of multiple flows serving a common objective. To capture these interactions, recent studies construct session-level graph representations. For instance, FG-Net [18] builds flow relation graphs connected by concurrent and sequential edges, utilizing Graph Attention Networks for application fingerprinting. Other hybrids integrate GNNs with Transformers to detect threats in smart grids or cloud environments [19].

However, these approaches face notable limitations. First, they often overlook auxiliary DNS/TLS metadata, limiting semantic characterization [7]. Second, they underexploit packet-level sequence features, often relying on convolutional or recurrent architectures that struggle with long-range dependencies. Third, most existing approaches rely on fixed-length time windows for session segmentation which can lead to information loss when flow arrival times are unevenly distributed [20].

To address these problems, *MV-FlowGraph* introduces four key innovations: (1) enrichment of flow behavioral features by DNS and TLS handshake fields; (2) burst-aware packet sequence transformation that explicitly accounts for bulk packet transmissions; (3) a dynamic sliding-window mechanism for more adaptive and fine-grained flow grouping when constructing flow relation graphs; and (4) a multi-view feature fusion framework that jointly integrates statistical features, packet-level sequences, protocol semantics, and inter-flow relations.

Notably, our *MV-FlowGraph* is not a simple ensemble of existing modules, but provides a comprehensive backbone for modeling traffic behaviors. Instead of directly processing raw payload bytes, it devotes extensive development efforts to correlate multiple complementary views of each flow to better characterize its behavior. It also extends existing flow-relation graph methods by considering both temporal adjacency among flows and relaxed sequential relations among bursts of flows.

III. An Overview of Framework Design

In this section, we propose (Multi-View) MV-FlowGraph, which adopts a multi-module framework, as shown in Fig. 2.

Intra-Flow Feature Extraction Module (Fig. 2(A)). We define a client-server session as a collection of flows initiated by a single client toward a specific server. Within each session, a client typically communicates with the server through multiple

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

five-tuple flows (*srcIP, srcPort, dstIP, dstPort, protocol*). To characterize each session, we first extract a comprehensive set of features for individual flows, including flow-level statistical features and raw IP packet sequences, which are standardized using the top-k most frequent packet lengths for noise suppression. To enrich features of an individual flow, we correlate its server IP to domain name from DNS traffic and represent it by application-domain co-occurrence embeddings derived from all flow labels. We also analyze TLS handshake metadata, such as Server Name Indication, cipher suites, and certificates.

Flow Relation Graph Construction Module (Fig. 2(B)). A client–server session may contain many flows spanning long time periods, potentially lasting hours or even days and carrying mixed behavioral labels. Therefore, after partitioning flows by client-server sessions, we further split each session into multiple contiguous activity periods. Specifically, flows within a session are first sorted by arrival time. Neighboring flows are grouped into the same burst if their inter-arrival time is below a threshold (e.g., 1 s), such that each burst captures tightly correlated concurrent flows occurring within a short interval. Adjacent bursts are then merged into the same session window when the average inter-arrival time after merging does not increase significantly. Each session window, consisting of one or more bursts, is modeled as a flow relation graph. In this graph, each vertex is a flow, which has multi-view features extracted by the previous module. Flows within the same burst are linked by concurrent edges, while flows across successive bursts are linked by sequential edges, as shown in Fig. 2(B).

Multi-View Feature Fusion Module (Fig. 2(C)). For multi-view feature fusion, we adopt a hierarchical two-layer embedding strategy. At the flow layer, DNS and TLS plaintext metadata are encoded by a pre-trained BERT model, flow-level statistical features are compressed via an autoencoder, and directed packet length sequences are modeled by a Transformer to capture temporal patterns [16]. The resulting view-specific embeddings are fused into a unified flow representation and assigned to graph vertices. At the session layer, the flow relation graph is processed by a Graph Attention Network to aggregate inter-flow dependencies and produce a session-window-level embedding for encrypted traffic classification.

IV. Extract Multi-View Features for Single Flows

In this section, we present the multi-view feature extraction module that covers the full lifecycle of a network flow, including domain name resolution via the DNS protocol, encryption negotiation during the TLS handshake, and subsequent encrypted payload transmission over TCP/IP. These complementary feature views characterize encrypted traffic from multiple perspectives, thereby improving the classification accuracy.

A. Domain Name Feature Extraction

Before initiating a network flow, a client typically issues a DNS query to resolve the server's domain name into an IP address. The resolved domain may correspond to a benign service (possibly under attack) or to a command-and-control

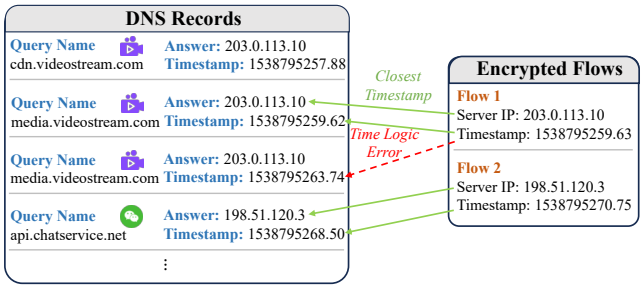


Fig. 3: Reverse retrieval of domain name from server IP.

(C&C) server. Thus, the domain name provides a stable application identifier, even when subsequent traffic is encrypted. For each flow, we infer the contacted server's domain name based on its server IP address and flow start time through a reverse DNS lookup procedure. Specifically, by online parsing DNS traffic, we extract plaintext A/AAAA records and maintain a database of reverse DNS records. Let DN_d denote the queried domain name, T_d the DNS query timestamp, A_d the set of resolved IP addresses, and TTL_d the corresponding time-to-live value. The reverse DNS database is defined as

$$\mathbf{D} = \{A_d \rightarrow (\mathrm{DN}_d, T_d, \mathrm{TTL}_d) \mid d \in \text{A/AAAA DNS records}\}.$$

Given a network flow f with server IP address A_f and start time T_f, we retrieve candidate DNS records from $\mathbf{D}$ that satisfy the following conditions: (1) $A_f \in A_d$, and (2) $T_d \leqslant T_f$. If multiple candidates exist, we select the record whose timestamp T_d is closest to T_f and further verify its validity using the associated TTL_d. The corresponding query name is then assigned as the domain name DN_f of flow f. A case study illustrating this reverse lookup process is given in Fig. 3.

After determining the domain name DN_f, we derive its representation using two complementary embedding strategies. First, the text of DN_f is encoded by a pre-trained BERT model to capture semantic patterns in domain names. Second, we construct a domain-application co-occurrence embedding based on access frequency statistics. Specifically, by scanning the training flow dataset, we build a domain-application co-occurrence matrix, in which the vector of occurrence frequencies associated with a particular domain $\mathrm{DN}i$ is defined as

$$\mathbf{e}_\mathbf{i}^{\mathrm{DN}} = [q_{1i}, q_{2i}, \ldots, q_{Ni}], \tag{1}$$

where q_{ji} denotes the frequency with which domain $\mathrm{DN}i$ is observed in traffic belonging to the j-th application class ($1 \leqslant j \leqslant N$). For a network flow f, its associated server domain name DN_f is embedded as $\mathbf{v}_{\mathrm{DN}_f} = \mathbf{e}_i^{\mathrm{DN}}$ if DN_f appears in the training set; otherwise, it is mapped to an all-zero vector. In practice, we retain only the first three levels of a domain name (e.g., sub.example.com), as deeper subdomains are sparsely observed and tend to produce near-zero co-occurrence vectors.

B. TLS Handshake Feature Extraction

After resolving the server's domain name, the TLS protocol is commonly used to establish an encrypted connection to the server IP address. Figure 4 illustrates its two phases: During

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

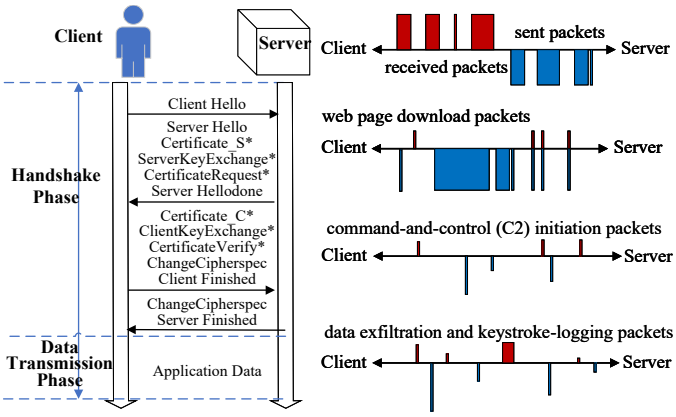


Fig. 4: TLS handshake and packet exchange of a connection.

the handshake phase, the client and server negotiate cryptographic algorithms, establish shared keys, and authenticate digital certificates [21]. In the subsequent data exchange phase, application-layer content is protected by encrypted ciphertext.

In TLS 1.2, handshake metadata provides a rich set of observable features for traffic classification. During the handshake, negotiated cryptographic parameters such as the *TLS version*, selected *cipher suite*, and *elliptic curve* are explicitly revealed. In addition, the `ClientHello` message discloses the client's supported *cipher suites*, *signature algorithms*, and *elliptic curves*. Handshake extensions further enrich the contextual information: The *server_name* (SNI) extension exposes the intended destination domain, while the Application-Layer Protocol Negotiation (ALPN) field indicates the negotiated application-layer protocol (e.g., HTTP/2 or QUIC). Server certificates reveal organizational and trust-chain attributes, including the certificate subject, issuer, and validity period. Collectively, these handshake features constitute critical observables for encrypted traffic analysis under TLS 1.2. In contrast, TLS 1.3 significantly reduces the visibility of handshake metadata due to enhanced encryption and privacy protections. Thus, many of the aforementioned TLS fields are no longer observable and are treated as `None` in our feature extraction.

C. Network Traffic Side-Channel Features

After the TLS handshake, packet payloads are encrypted, but side-channel features remain observable. We extract two types of features: per-flow statistics (e.g., minimum, maximum, average, and sum of packet lengths and inter-packet times), and time-series features such as directed sequences of packet lengths and arrival time intervals. More specifically:

- **Per-flow statistical features:** The statistics capture the volumetric and temporal features of each flow, including mean, minimum, maximum, and standard deviation of packet numbers, payload sizes, and inter-arrival times in client-to-server and server-to-client directions, with dimension $d_{\text{stat}} = 103$.
- **IP packet direction/size/IAT sequences:** Packet sequence features, such as directions, payload sizes, and inter-arrival times (IAT) extracted from TCP/IP headers and ordered by timestamp, offer strong discriminative power for predictive models [6]. The right side of Fig. 4 depicts three typical packet transmission patterns: (1) normal webpage download, (2) command-and-control heartbeat signal exchange, and (3) data exfiltration. These examples demonstrate how packet-level features can effectively capture network behaviors.

Later, we will apply a Transformer-based encoder to these IP packet sequences, which is more effective than recurrent neural networks in capturing long-range dependencies. However, in practice, the lengths of packet sequences vary significantly across different flows. Due to the limited processing capacity of Transformer models, overly long sequences must be truncated to a fixed length (e.g., 512 packets), which inevitably leads to information loss. To alleviate this issue, we propose to compress packet sequences by aggregating consecutive *bulk-transmitted* packets into *large-payload virtual packets*. A bulk transmission is defined as a continuous sequence of payload-carrying packets sent in the same direction (either from the source or from the destination), with a minimum length of `bulk_min_length` (default: 5 packets). A bulk transmission is terminated when either (i) a payload-carrying packet is observed in the opposite direction, or (ii) the inter-arrival time between two consecutive packets exceeds a timeout threshold `bulk_timeout` (default: 1 second). If, upon termination, the number of packets in the bulk transmission is smaller than `bulk_min_length`, the corresponding bulk is discarded.

V. Embed Multi-View Features of Individual Flows

For each single network flow, we extract three categories of features, each represented using a dedicated encoding scheme:

- **Tabular view** consists of flow statistical features augmented with domain-app co-occurrence features derived from server domain names. These features are encoded by an autoencoder.
- **Packet-sequence view** captures fine-grained IP packet sequence features of a flow through directed payload-size sequences and inter-packet arrival time (IAT) sequences.
- **Text view** is semantic information of high-level protocols, including DNS query strings, TLS handshake metadata, and X.509 certificates, all of which are semi-structured texts.

A. Tabular View with Flow Statistical and Domain Features

Let $\mathbf{x}f^{\text{tab}}$ denote the tabular feature vector of flow f, formed by concatenating the normalized flow-level statistical features ($d_{\text{stat}} = 103$) and the domain name embedding defined in Eq. (1). This vector is fed into the tabular feature embedding module. As illustrated in Fig. 2(C1), we adopt an autoencoder to learn a compact and discriminative representation of the tabular features. Specifically, the encoder transforms $\mathbf{x}_f^{\text{tab}}$ into a latent embedding through a series of nonlinear operations:

$$\mathbf{d}_f^{\text{tab}} = \text{Encoder}(\mathbf{x}_f^{\text{tab}}) = \text{Attn}\left(\text{ReLU}\left(\text{LN}\left(\mathbf{W}_e\mathbf{x}_f^{\text{tab}} + \mathbf{b}_e\right)\right)\right),$$

where $\mathbf{W}_e$ and $\mathbf{b}_e$ are trainable encoder parameters, $\text{LN}(\cdot)$ denotes layer normalization, and $\text{Attn}(\cdot)$ applies a feature-wise attention mechanism to emphasize salient dimensions.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

The decoder mirrors the structure of the encoder and reconstructs the input feature vector as $\hat{\mathbf{x}}_f^{\text{tab}}$, enabling self-supervised learning that preserves essential information in the latent space. The autoencoder is trained by minimizing the mean squared reconstruction error over the set of all flows F:

$$\mathcal{L}_{\text{AE}} = \frac{1}{|F|} \sum_{f \in F} \left\| \hat{\mathbf{x}}_f^{\text{tab}} - \mathbf{x}_f^{\text{tab}} \right\|_2^2, \text{ with } \hat{\mathbf{x}}_f^{\text{tab}} = \text{Decoder}(\mathbf{d}_f^{\text{tab}}).$$

The tabular feature embedding is obtained by applying a fully connected transformation to the attended latent representation:

$$\mathbf{z}_f^{\text{tab}} = \text{ReLU}\big(\text{LN}(\text{Linear}_{\text{tab}}(\mathbf{d}_f^{\text{tab}}))\big), \tag{2}$$

where $\mathbf{z}_f^{\text{tab}}$ is the embedding of the tabular features for flow f.

B. Virtual IP Packet Sequence Features Modeling

For the "virtual packet" sequence of a flow f, let $d_{f,t} \in \{+1, -1\}$, $x_{f,t}^{\text{len}}$ and $x_{f,t}^{\text{iat}}$ be the direction, payload size and inter-arrival time at packet position t, respectively. The sequences have been padded or truncated to a fixed length T (default: $T = 512$). Prior to sequence encoding, the raw signals are normalized to improve training stability. Specifically, the payload size $x_{f,t}^{\text{len}}$ is divided by the maximum transmission unit (MTU) and multiplied by the packet direction $d_{f,t}$, while the inter-arrival time $x_{f,t}^{\text{iat}}$ is transformed by a log-scale normalization function to mitigate its highly skewed distribution:

$$\tilde{x}_{f,t}^{\text{len}} = d_{f,t} \cdot x_{f,t}^{\text{len}} / \text{MTU}, \qquad \tilde{x}_{f,t}^{\text{iat}} = \log\left(1 + (x_{f,t}^{\text{iat}} + \epsilon) / \tau\right),$$

where ε is a small constant for numerical stability; τ is a scale parameter (in seconds) controlling the normalization range.

To extract sequential patterns and long-range dependencies, the two normalized scalars $\tilde{x}_{f,t}^{\text{len}}$ and $\tilde{x}_{f,t}^{\text{iat}}$ are independently projected into separate embedding spaces $\mathbb{R}^d$ by linear mappings. The resulting embeddings are combined with a learnable positional embedding $\mathbf{p}_t \in \mathbb{R}^d$ to form the final token embedding:

$$\mathbf{h}_{f,t} = \alpha_1 \cdot \text{Linear}_{\text{len}}\big(\tilde{x}_{f,t}^{\text{len}}\big) + \alpha_2 \cdot \text{Linear}_{\text{iat}}\big(\tilde{x}_{f,t}^{\text{iat}}\big) + \mathbf{p}_t, \tag{3}$$
$$\text{where} \quad [\alpha_1, \alpha_2] = \text{Softmax}([\lambda_{\text{len}}, \lambda_{\text{iat}}]).$$

This *adaptive fusion* allows the model to automatically balance the relative importance of payload size and IAT information.

The embedded sequence $(\mathbf{h}_{f,1}, \ldots, \mathbf{h}_{f,T})$ is processed by a stack of Transformer encoder layers, as shown in Fig. 2(C2):

$$\mathbf{H}_f = \text{Pre-LayerNorm-TransformerEncoder}(\mathbf{h}_{f,1:T}), \tag{4}$$

where self-attention models contextual dependencies within packet sequences, and the Pre-LayerNorm architecture applies layer normalization before the multi-head self-attention and feed-forward sub-layers, in order to stabilize the training of deep sequence encoders on long packet sequences. To obtain a sequence-level representation, we use a learnable query vector $\mathbf{q}$ to perform attention pooling over the encoded sequence.

$$\mathbf{z}_f^{\text{seq}} = \text{AttnPool}\left(\mathbf{q}, \mathbf{H}_f, \mathbf{H}_f\right) \tag{5}$$

C. Fusion of Multi-View Flow-Level Features

Our model extracts three complementary feature views for each network flow f: (1) a tabular view $\mathbf{z}_f^{\text{tab}}$, (2) a sequence view $\mathbf{z}_f^{\text{seq}}$, and (3) a text view $\mathbf{z}_f^{\text{text}}$, constructed from protocol-level semantic fields, including DNS query names, TLS Server Name Indication, and X.509 certificates. These fields are concatenated as a combined text and embedded by a pre-trained BERT model, as illustrated in Fig. 2(C3). The combined text representation is obtained from the `[CLS]` token embedding.

$$\mathbf{z}_f^{\text{text}} = \text{BERTEncoder}\left(\text{combined textual features of flow } f\right)$$

To derive a unified flow-level representation, the three view embeddings are concatenated and projected via a fusion layer.

$$\mathbf{z}_f^{\text{multi}} = \text{ReLU}\big(\text{LN}(\text{Linear}_{\text{fuse}}(\big[\mathbf{z}_f^{\text{tab}} \,\big|\, \mathbf{z}_f^{\text{seq}} \,\big|\, \mathbf{z}_f^{\text{text}}\big]))\big) \tag{6}$$

In addition to simple view concatenation, the fusion layer can alternatively be implemented using weighted summation or cross-attention mechanisms to model inter-view dependencies.

VI. Multi-Flow Behavior of Client-Server Session

A client often initiates multiple network flows toward the same server within a short time interval, either to retrieve online content or to conduct malicious activities. Consequently, a single flow is typically insufficient to fully characterize the behavior of a client–server session. Instead, the collection of flows belonging to the same session conveys richer behavioral semantics and can be represented as a graph structure, as illustrated in Fig. 2(B). In this section, we construct flow relation graphs to capture concurrent and sequential relations among multiple flows exchanged between communicating parties, and use graph neural networks to model session-level behaviors.

A. Flow Relation Graph (FRG) Construction

First, we partition flows according to client-server sessions, as shown in Fig. 2(A). Then, for each partition, flows are sorted by their start timestamps, after which the Flow Relation Graph (FRG) is constructed by three major stages: burst clustering, multi-flow window aggregation, and graph topology building.

1. Burst Clustering: Identifying Concurrent Flow Groups. The goal of burst clustering is to group temporally adjacent flows, called *concurrent flows*, within the same client-server session. Such flows typically correspond to a single short-term application activity, for example, consecutive HTTPS requests issued during a DoS attack to overwhelm a target server. To identify burst boundaries, we introduce a burst time threshold δ, which is calibrated according to application characteristics. For low-latency applications (e.g., video streaming), δ is set to $500\,\text{ms}$ to capture rapid successive resource requests. In contrast, for delay-tolerant applications (e.g., file transfer), δ is extended to $2\,\text{s}$ to accommodate larger inter-flow time gaps.

2. Multi-Flow Session Window Aggregation. A client–server interaction may span multiple *session windows*, each of which aggregates temporally adjacent bursts into a higher-level unit that better aligns with application communication processes. The procedure is guided by the average inter-flow interval.

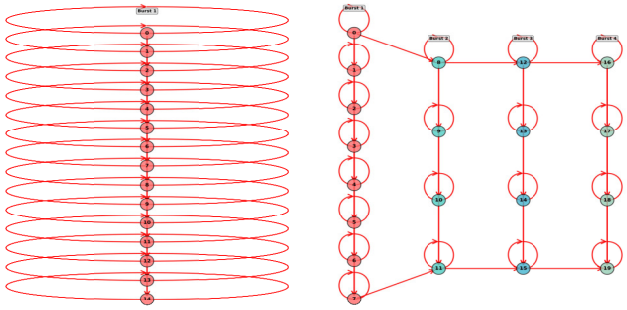
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

- **Initialization:** The first burst within each client-server session is used to start a new session window. If the window has a single flow, the average inter-flow interval is defined as δ. Otherwise, it is computed as $\frac{\text{Max Start Time - Min Start Time}}{\text{Number of Flows - 1}}$.
- **Burst Merging Criterion:** Let $\overline{\Delta t}_{i,j}$ be the average inter-flow interval for neighboring flows within a candidate session window, which carry indices ranging from i to j. This quantity serves as a heuristic measure of application-level temporal cohesion. A new burst is merged into the current session window only if its inclusion does not significantly inflate the average inter-flow interval, thereby preserving temporal compactness. Specifically, the merge condition is $\overline{\Delta t}_{i,j} < \alpha\,\overline{\Delta t}_{i,k}$, where k is the index of the last flow currently in the session window with $k < j$, and α is a relaxation factor, set to 1 by default. An example is shown in Fig. 2(B): Burst 2 is merged with the ongoing session containing burst 1 since the average interval of flows 1–3 remains bounded by α times that of flows 1–2, i.e., $\overline{\Delta t}_{1,3} \leqslant \alpha\,\overline{\Delta t}_{1,2}$. Similarly, burst 3 is merged since flows 1–6 form a temporally dense cluster satisfying $\overline{\Delta t}_{1,6} \leqslant \alpha\,\overline{\Delta t}_{1,3}$. In contrast, burst 9 is excluded as its inclusion causes a great increase in average inter-flow interval, i.e., $\overline{\Delta t}_{1,9} > \alpha\,\overline{\Delta t}_{1,8}$.
- **Session Window Output:** Each finalized session window is output with a unique ID and a time-sorted list of bursts.

3. Flow Relation Definition and Graph Topology Construction. We build a flow-relation graph for each session window, to model concurrent & sequential interactions among its flows.

- **Vertex Attributes:** Each vertex corresponds to a network flow f and stores its start timestamp, flow-level statistical features, packet-level temporal sequences, as well as DNS and TLS metadata as illustrated in Fig. 2(B). These features are represented and fused as a multi-view embedding z_f^{multi}.
- **Edge Definition:** Two types of edges are constructed in the flow relation graph. *Concurrent edges* connect flows within the same burst to capture temporal concurrency. We support three concurrent edge construction strategies, including *full-connect*, *chain-connect*, and *k-nearest* based on temporal proximity; by default, we use the k-nearest strategy with $k = 3$, to balance structural expressiveness and graph sparsity. A *sequential edge* connects the first/last flow of one burst to the first/last flow of the subsequent burst, thereby modeling the sequential relation between consecutive bursts. In addition, each node is augmented with a self-loop concurrent edge to enhance the training stability of the GNN model.

To demonstrate the capability of our MV-FlowGraph in capturing diverse attack patterns, Figure 5 visualizes the FRGs of two representative attack types under $\delta = 1\,\text{s}$. Figure 5(a) shows an SSH-Patator session, which exhibits a *single-burst, high-concurrency topology*. This dense vertical structure arises because the attacker launches rapid, parallel login attempts within a short window to guess credentials, resulting in strong intra-burst connectivity. In contrast, Fig. 5(b) depicts a DoS Slowloris session with a *multi-burst, sequential topology*. Unlike brute-force attacks, Slowloris adopts a "low-and-slow" strategy, periodically sending partial headers to maintain connections. These intentional delays exceed δ, segmenting the session into multiple bursts linked by inter-burst edges. These distinct structural differences confirm that our FRG construction effectively encodes the unique temporal semantics of both high-rate and slow-rate attack types. Compared with conventional flow-level intrusion detectors, the session-window-level FRG-based detector better models coordinated malicious behaviors while reducing redundant security alerts.



(a) SSH-Patator (b) DoS Slowloris

Fig. 5: Compare flow relation graphs of two different attacks.

B. Flow Relation Graph Representation and Classification

For each session window, the fused flow embeddings z_f^{multi} are treated as vertex attributes and propagated over the corresponding flow relation graph by a graph neural network. Specifically, we convert the heterogeneous flow relation graph into a homogeneous graph and adopt a multi-layer GATv2 [22] to perform message passing among flows. It uses dynamic attention to capture both concurrent and sequential inter-flow relations more effectively than standard static attentions.

Let $\mathbf{H}_s = \{\mathbf{h}_f^{(L)} \mid f \in s\}$ denote the set of node embeddings produced by the graph encoder for session window s after L layers of message passing. A permutation-invariant readout operation is then applied to aggregate node-level representations into a fixed-dimensional session-level embedding. In our implementation, it is realized by mean pooling over all nodes:

$$\mathbf{z}_s^{\text{global}} = \tfrac{1}{|\mathbf{H}_s|} \textstyle\sum_{\mathbf{h}_f^{(L)} \in \mathbf{H}_s} \mathbf{h}_f^{(L)}, \;\; \text{with } \mathbf{h}_f^{(L)} = \text{GNN}(f, \mathcal{N}(f)) .$$

The node representations are obtained by iterative graph message passing: $\mathcal{N}(f)$ is the neighborhood of flow f in the graph, $\mathbf{h}_f^{(0)} = z_f^{\text{multi}}$ is the initial node embedding, and $\mathbf{h}_f^{(L)}$ is the updated node embeddings after L layers of message passing.

The resulting global session representation $\mathbf{z}_s^{\text{global}}$ is shared by multiple task-specific classification heads, corresponding to hierarchical intrusion detection tasks at different granularities. We perform the coarse-grained malicious session detection, as well as fine-grained attack-family & attack-type classification:

$$\hat{y}_s^{\text{mal}} = \sigma\big(\mathbf{W}_{\text{mal}}\, \mathbf{z}_s^{\text{global}} + \mathbf{b}_{\text{mal}}\big) , \tag{7}$$

$$\hat{\mathbf{y}}_s^{\text{fam}} = \sigma\big(\mathbf{W}_{\text{fam}}\, \mathbf{z}_s^{\text{global}} + \mathbf{b}_{\text{fam}}\big) , \tag{8}$$

$$\hat{\mathbf{y}}_s^{\text{type}} = \text{Softmax}\big(\mathbf{W}_{\text{type}}\, \mathbf{z}_s^{\text{global}} + \mathbf{b}_{\text{type}}\big) . \tag{9}$$

Here, $\sigma(\cdot)$ is the sigmoid function. The attack-family prediction is formulated as a one-vs-rest (OVR) multi-label classifi-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

cation task. The attack-type prediction is modeled as a multi-class classification task with a softmax output. Attack families and types are defined for the CIC-IDS-2017 dataset in Table I.

The overall training objective is defined as a weighted sum:

$$\mathcal{L} = \lambda_{\text{mal}}\mathcal{L}_{\text{mal}} + \lambda_{\text{fam}}\mathcal{L}_{\text{fam}} + \lambda_{\text{type}}\mathcal{L}_{\text{type}}, \tag{10}$$

where λ_{mal}, λ_{fam}, and λ_{type} are task-specific weighting coefficients. The malicious session detection loss $\mathcal{L}_{\text{mal}}$ is implemented as a weighted binary cross-entropy over all sessions. The attack-family loss $\mathcal{L}_{\text{fam}}$ is defined as a one-vs-rest binary cross-entropy loss and is computed only on sessions labeled as malicious to ensure hierarchical consistency. For the highly imbalanced attack-type classification task, we employ the focal loss [23], which is also computed only on malicious sessions.

VII. Experiments

We conduct experiments to evaluate MV-FlowGraph's performance in classifying encrypted traffic. We first describe the datasets and experimental setup, followed by a comprehensive analysis of the results, including comparisons with baseline methods and ablation studies of components in our model.

A. Experimental Settings

Comparison Methods: Several recent methods for mobile app fingerprinting are summarized as baselines for comparison:

- **AppScanner** [10] extracts 54 statistical features (e.g., packet count, mean packet size, inter-arrival time) from each flow and trains a random forest classifier to identify mobile apps.
- **FS-Net** [6] employs a GRU-based autoencoder to classify traffic using directed packet length sequences.
- **FG-Net** [18] builds a *flow relationship graph*, applying graph attention to packet-level features to capture inter-flow concurrent and sequential correlations.
- **TrafficFormer** [17] uses self-attention to capture temporal dependencies among packets within each flow, processing them independently without inter-flow topology modeling.

Performance Metrics. To evaluate the effectiveness of the aforementioned models in detecting multi-category malicious encrypted traffic, we use macro-averaged accuracy, precision, recall, and F1-score as evaluation metrics, as defined below:

$$\text{Macro Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\frac{\text{TP}_i+\text{TN}_i}{\text{TP}_i+\text{TN}_i+\text{FP}_i+\text{FN}_i}, \tag{11}$$

$$\text{Macro Precision} = \frac{1}{N}\sum_{i=1}^{N}\frac{TP_i}{TP_i+FP_i}, \tag{12}$$

$$\text{Macro Recall} = \frac{1}{N}\sum_{i=1}^{N}\frac{TP_i}{TP_i+FN_i}, \tag{13}$$

$$\text{Macro F1} = \frac{2\times\text{Macro Precision}\times\text{Macro Recall}}{\text{Macro Precision}+\text{Macro Recall}}. \tag{14}$$

Here, N is the number of malicious traffic categories. For each class i, TP_i, FP_i, FN_i, and TN_i are the numbers of true positives, false positives, false negatives, and true negatives, respectively. Macro-averaging assigns equal weight to each class and thus provides a fair assessment under class imbalance.

Model Configuration. Our MV-FlowGraph has a packet-sequence Transformer (4 layers, 8 heads, 256-dim, max length 256), a compact pre-trained BERT (2 layers, 128-dim) for text, and a GATv2 graph encoder (3 layers, 8 heads, 128-dim). Default burst parameters are $\delta = 1\,\text{s}$ and $\alpha = 1.0$. All models are trained by PyTorch Lightning on 2 NVIDIA RTX 3090 GPUs. Each model is trained for up to 40 epochs with the AdamW optimizer, using a learning rate of 1×10^{-6} and a batch size of 32. The training/validation/test sets are randomly partitioned in an 80:10:10 ratio. We apply gradient clipping with a max norm of 1.0 and early stopping based on validation Macro-F1 with a patience of 8 epochs to improve stability.

B. Analyze Dataset Properties and Configure Class Weights

We conducted experiments on the CIC-IDS-2017 and CIC-IDS-2018 datasets. These two datasets are widely adopted in the network intrusion detection community, since they encompass a wide spectrum of benign and malicious traffic scenarios.

TABLE I: Class distr. and loss weights of CIC-IDS-2017.

Family	Neg/Pos Wt.	Attack Type	Ratio	Focal Wt.
Normal	[1.04, 1.0]	Benign	51.12%	—
DoS	[1.0, 1.35]	DoS Hulk	17.85%	1.00
		DoS Slowhttptest	1.76%	3.18
		DoS GoldenEye	0.83%	4.63
		DoS slowloris	0.42%	6.48
Probe	[1.0, 1.81]	PortScan	17.42%	1.01
DDoS	[1.0, 4.17]	DDoS	9.47%	1.37
Brute Force	[1.0, 63.2]	FTP-Patator	0.44%	6.40
		SSH-Patator	0.33%	7.40
		Web Attack-Brute Force	0.15%	9.00
Botnet	[1.0, 201.5]	Bot	0.24%	8.60
Total	–		**99.73%**	–

† Rare attacks such as Web Attack–XSS, SQL Injection, Infiltration, and Heartbleed are excluded from training due to limited samples.

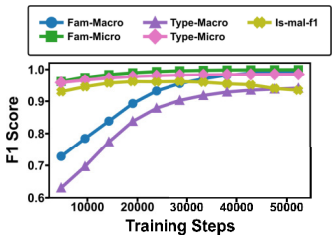
As shown in Table I, CIC-IDS-2017 exhibits severe long-tailed class imbalance. While the binary distribution is relatively balanced, specific attacks like Botnet are extremely rare, which risks biasing models toward dominant classes. CIC-IDS-2018 presents a more extreme skewness, with 99.1% benign traffic extracted from 1 TB of raw pcaps, posing significant challenges for robust minority attack detection.

To address this issue, we employ a hierarchical loss reweighting strategy aligned with task granularity. Specifically, Neg/Pos weights are applied to the binary cross-entropy loss for coarse-grained malicious traffic detection. For attack-family classification (one-vs-rest), Neg/Pos weights are computed on the malicious subset to balance family-level frequencies. At the finest granularity, we adopt a class-weighted (e.g., 7.40 for SSH-Patator) focal loss to down-weight dominant classes and emphasize hard, minority attack types.

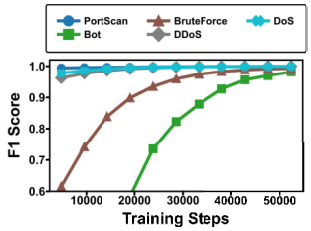
C. Training Convergence & Multi-Class Classification Results

Figure 6 illustrates the training convergence of MV-FlowGraph on the CIC-IDS-2017 dataset under different classification granularities, where sessions are defined using five-tuple identifiers. **(a)** shows the evolution of F1 scores for the *is_malicious* detection task, which converges rapidly and

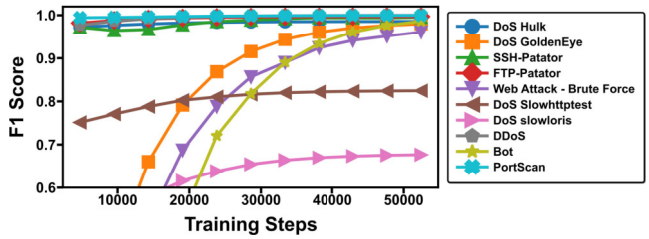
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.



(a) F1 scores for is_malicious detection and attack family/type classification (macro and micro F1).



(b) Training convergence of F1 scores across different coarse-grained attack-family classes.



(c) Training convergence of F1 scores for different attack types.

Fig. 6: Performance of MV-FlowGraph on CIC-IDS-2017.

achieves a high F1 score at an early training stage. Plot **(a)** also reports the F1 scores for attack family and attack type classification tasks (as defined in Table I) using both macro- and micro-averaged metrics. The micro-F1 scores consistently exceed the macro-F1 scores due to class imbalance, while the macro-F1 scores gradually improve and stabilize, indicating enhanced discrimination capability for minority classes.

Next, in subfigure **(b)**, we present the convergence of F1 scores for different attack families. Most families, including *PortScan*, *DDoS*, and *DoS*, converge quickly to near-optimal performance, showing effective modeling of coarse-grained behavioral patterns. In contrast, *BruteForce* and *Bot* show slower convergence, likely due to higher intra-class variability.

Finally, **(c)** depicts the convergence for fine-grained attack types, revealing more learning dynamics. While several attack types (e.g., *DoS Hulk*, *PortScan*, and *DDoS*) reach near-perfect F1 scores early, others (e.g., *DoS slowloris* and *DoS Slowhttptest*) converge more slowly and stabilize at lower F1 values, reflecting the difficulty of separating these two attacks.

As shown in Table II, MV-FlowGraph achieves a state-of-the-art F1-score of 99.57%. It surpasses traditional methods (AppScanner, FS-Net) via holistic multi-view fusion. Compared to the graph-based FG-Net, our model benefits from Transformer sequence encoding and rich semantic metadata. Crucially, it outperforms the strong sequence baseline TrafficFormer (96.48% F1) by leveraging flow relation graphs, proving that integrating intra-flow dynamics with inter-flow macro-topology is essential for detecting complex intrusions.

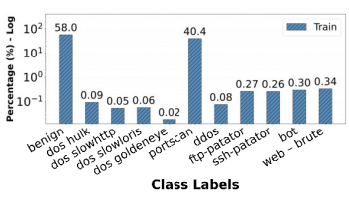
D. Flow Relation Graphs for Session Keys without srcPort

We define session keys as srcIP–dstIP–dstPort–proto tuples, and depict the characteristics of CIC-IDS-2017 and CIC-IDS-2018 datasets in Fig. 7. Both exhibit extreme class imbalance. While the former has more attack types, the latter is heavily

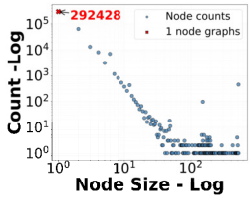
TABLE II: Accuracy, Precision and F1 comparison of different methods for detecting network intrusions in CIC-IDS-2017.

Methods	Accuracy (%)	Precision (%)	F1 (%)
Appscanner [10]	93.42	92.08	93.50
FS-Net [6]	89.52	85.50	90.04
FG-Net [18]	90.10	85.73	90.63
TrafficFormer [17]	95.49	94.96	96.48
MV-FlowGraph	**99.64**	**99.43**	**99.57**

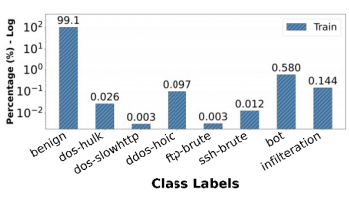
skewed with benign traffic at 99.1%. Such sparsity requires to detect rare attacks like Bot (0.58%) and Infiltration (0.14%).



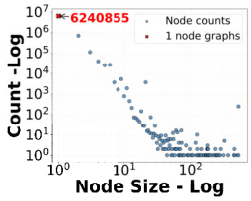
(a) Label distribution (IDS-2017)



(b) Node # distr. (IDS-2017)



(c) Label distribution (IDS-2018)



(d) Node # distr. (IDS-2018)

Fig. 7: Data distribution analysis of the datasets.

Figures 7b and 7d show that session graph sizes follow a power-law distribution. While most graphs represent simple interactions, the heavy tail reveals complex multi-flow attacks (e.g., DDoS) reaching $10^3 \sim 10^4$ nodes, which risk causing GPU out-of-memory errors. To ensure training efficiency, our capacity-aware session window truncates the maximum node count to 512, effectively bounding memory consumption.

E. Ablation Experiments and Results

Table III shows that the full MV-FlowGraph model achieves the best binary F1-score of 99.57%. Removing the packet sequence encoder causes the most significant drop, highlighting the importance of fine-grained temporal patterns. Excluding textual features or the flow relation graph reduces F1 to 95.78% and 96.42%, respectively. Notably, the variant without textual features achieves a higher Macro F1 for attack type classification. This is because shared textual signatures across attacks targeting the same server introduce ambiguity, whereas statistical features are more discriminative for specific attack behaviors. However, textual features are retained as they significantly boost the primary binary detection performance.

To evaluate deployment feasibility, we benchmarked MV-FlowGraph's efficiency on an RTX 3090 GPU (Table IV). The full model needs only 9.36M parameters and 142.54MB VRAM, attaining a sub-millisecond latency (0.34ms) and high throughput (3,000 sessions/s). Ablation reveals that deep feature extractors dominate memory costs; removing the sequence encoder reduces VRAM to 59.00MB. Thus, our GNN module

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Ablation study of different components in MV-FlowGraph. "w/o Deep Features" denotes the model using only flow statistical features (without text and sequence views).

	Binary Classification(is_mal)			Attack Type
Models	**Precision**	**Recall**	**F1**	**F1**
MV-FlowGraph (full)	**99.43%**	**99.71%**	**99.57%**	79.40%
w/o Textual Features	93.75%	97.90%	95.78%	**91.74%**
w/o Packet Sequence	88.70%	96.97%	92.65%	69.99%
w/o Deep Features(stats only)	89.85%	92.04%	90.93%	52.24%
w/o Flow Relation Graph	93.81%	98.83%	96.42%	77.23%

TABLE IV: Efficiency & computational cost per session graph.

Model Variant	Params(M)	VRAM(MB)	Latency(ms)	Thpt.(Sess/s)
MV-FlowGraph (full)	**9.36**	**142.54**	**0.34**	**2973.91**
w/o Textual Features	9.00	140.78	0.26	3914.29
w/o Packet Sequence	9.30	59.00	0.20	4976.04
w/o Deep Features(stats only)	8.93	43.60	0.11	8993.76
w/o Flow Relation Graph	9.23	142.03	0.30	3303.03

incurs negligible overhead over the single-flow baseline, while providing flow topology awareness for multi-flow modeling.

Table V shows that each feature view contributes complementary information. The full model achieves the highest F1-score of 99.57%, outperforming all ablated variants. Specifically, removing SSL/TLS features causes a notable decline, verifying the discriminative power of unencrypted negotiation metadata. Excluding domain names or inter-packet arrival times (IAT) also degrades performance to 96.01% and 96.42%, respectively. Furthermore, relying solely on basic flow statistical features yields only 93.48%, which strongly validates our strategy of combining statistical, temporal, and protocol-specific features for comprehensive traffic characterization.

TABLE V: Ablation results of combinations of flow features.

Models	Accuracy (%)	F1 (%)
MV-FlowGraph (full)	**99.64**	**99.57**
Removing inter-packet IAT features	96.33	96.42
Removing domain features	95.93	96.01
Removing X509 features	96.04	96.14
Removing SSL features	95.69	95.75
Only flow statistical features	93.40	93.48

VIII. Conclusion

In this paper, we propose *MV-FlowGraph*, a multi-view flow representation and relation graph modeling framework for encrypted network traffic analysis. MV-FlowGraph integrates heterogeneous flow-level features, including packet-level temporal sequences, DNS and TLS metadata, and inter-flow relation graphs, through a hybrid Transformer–GNN-based feature fusion model. By jointly modeling intra-flow multi-view features and inter-flow dependencies, the proposed framework addresses key limitations of existing coarse-grained approaches, such as insufficient utilization of protocol semantics and the lack of effective modeling of multi-flow interactions. Extensive experiments on public benchmark datasets demonstrate that MV-FlowGraph enables more expressive representations of encrypted network traffic and outperforms state-of-the-art traffic analysis methods under diverse network environments.

Acknowledgement

This work was supported in part by National Natural Science Foundation of China under Grant 62372106, by a grant from Nanjing Sinovatio Technology Corporation, and by the Big Data Computing Center of Southeast University.

References

[1] R. Boutaba, M. A. Salahuddin *et al.*, "A comprehensive survey on machine learning for networking and communications," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3381–3432, 2018.

[2] K. Bathiri and M. Vijayakumar, "Enhancing intrusion detection system through DPI with machine learning approaches," *IEEE ADICS*, 2024.

[3] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, and Y. Liu, "Encrypted traffic classification of decentralized applications on ethereum using feature fusion," in *Proc. of IEEE/ACM IWQoS*, 2019, pp. 1–10.

[4] C. Lin *et al.*, "Trage: A generic packet representation for traffic classification based on header-payload differences," *IEEE/ACM IWQoS*, 2025.

[5] R. Alshammari and A. N. Zincir-Heywood, "Can encrypted traffic be identified without port numbers, ip addresses and payload inspection?" *Computer Networks*, vol. 55, no. 6, pp. 1326–1350, 2011.

[6] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," in *IEEE INFOCOM*, 2019.

[7] Z. Weng, T. Chen, T. Zhu, H. Dong, D. Zhou, and O. Alfarraj, "TLSmell: Direct identification on malicious https encryption traffic with simple connection-specific indicators." *CSSE*, vol. 37, no. 1, pp. 105–119, 2021.

[8] J. Lokoč, J. Kohout, P. Čech, T. Skopal, and T. Pevnỳ, "k-NN classification of malware in https traffic using the metric space approach," in *Proc. of Springer PAISI*, 2016, pp. 131–145.

[9] R. Dai, C. Gao, B. Lang, L. Yang, H. Liu, and S. Chen, "SSL malicious traffic detection based on multi-view features," in *ACM ICCNS*, 2019.

[10] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. of IEEE EuroS&P*, 2016, pp. 439–454.

[11] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, and B. Thuraisingham, "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. of ACM ACSAC*, 2016.

[12] G. CHENG, G. ZHANG, and Q. SHANG, "A semi-supervised method for unknown protocol identification based on constraint sets," *Journal of Southeast University (Natural Science Edition)*, vol. 56, no. 3, 2026.

[13] W. Zhang, X. Meng, J. Li, Y. Wang, and Y. Zhang, "Multi-layer collaborative bandit for multivariate time series anomaly detection," in *Proc. of IEEE/ACM IWQoS*, 2023, pp. 1–10.

[14] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. of ACM CCS*, 2018, pp. 1928–1943.

[15] G. Yonghao, X. Hao, and Z. Xiaoqing, "Encrypted malicious traffic detection based on multi-granularity representation learning," *Chinese Journal of Computers*, vol. 46, no. 9, pp. 1888–1899, 9 2023.

[16] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. of ACM WWW*, 2022.

[17] T. Zhao, L. Fang, X. Ma, X. Li, and C. Zhang, "TFformer: A time–frequency domain bidirectional sequence-level attention based transformer for interpretable long-term sequence forecasting," *Pattern Recognition*, vol. 158, p. 110994, 2025.

[18] M. Jiang, Z. Li, P. Fu, W. Cai, M. Cui, G. Xiong, and G. Gou, "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Computer Networks*, vol. 217, p. 109309, 2022.

[19] V. Govindarajan and J. H. Muzamal, "Advanced cloud intrusion detection framework using graph based features transformers and contrastive learning," *Scientific Reports*, vol. 15, no. 1, p. 20511, 2025.

[20] W. Li, X.-Y. Zhang, H. Bao, B. Yang, Z. Li, H. Shi, and Q. Wang, "Prism: Real-time privacy protection against temporal network traffic analyzers," *IEEE TIFS*, vol. 18, pp. 2524–2537, 2023.

[21] X. Yang, S. Ruan, Y. Yue, and B. Sun, "PETNet: Plaintext-aware encrypted traffic detection network for identifying cobalt strike https traffics," *Elsevier Computer Networks*, vol. 238, p. 110120, 2024.

[22] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. of ICLR*, 2022.

[23] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE TPAMI*, vol. 42, no. 2, 2020.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:35:38 UTC from IEEE Xplore. Restrictions apply.